Refund / Upgrade / Downgrade — Implementation Plan

Purpose: Close the payment-side loop for ticket tier changes and cancellations so admins stop bouncing to the PayMongo dashboard for routine work. Ordered by leverage: build the one primitive that unlocks the rest, then compose it.

Prerequisite: Confirm the PayMongo Refunds API is enabled on your merchant account before starting step 1. Everything else can start in parallel.

Where we are today

Already shipped

- **Refunds table** and admin queue at `/admin/refunds`. Rows carry entity_type + entity_id + amount + status + audit trail.
- **Tier change API** at `POST /api/events/tier-change`. Three cases: same-price (instant swap), upgrade (PayMongo checkout for delta), downgrade (pending refund row with a machine-readable marker in the note).
- **Member self-service UI** — `TierChangeBlock` component on `/members/tickets/[id]`. Guards: owner, active ticket, non-bundle, at least 12h before event, tier slot available.
- **Webhook `tier_upgrade` handler** — on payment success, reads `new_tier_id` from the stored transaction metadata and updates the ticket's `tier_id`.
- **Downgrade completion handler** — when an admin marks an event_ticket refund complete, the `[tier_change_target:UUID]` marker in the note triggers a tier swap instead of a ticket cancel.
- **Waitlist auto-promote helper** — `promoteNextWaitlist(eventId, n)` available as a reusable library function.

Migration outstanding

`supabase/migrations/20260905_refunds_event_ticket.sql` widens the `refunds.entity_type` CHECK constraint to accept `'event_ticket'`. Not yet applied on the remote database. Blocks the downgrade path only — upgrades and same-price swaps work without it.

Step 1 — PayMongo auto-refund

Why first: every downstream feature reuses this endpoint. Without it, downgrades and ticket cancels still require the admin to open PayMongo's dashboard.

Prerequisite check: is the Refunds API enabled on your merchant account? Ask PayMongo support:

```
Subject: Enable Refunds API for our account
Merchant ID: [your PayMongo merchant id]

Please enable the /v1/refunds endpoint for our account,
including QR Ph payment sources. We need to process refunds
programmatically from our admin dashboard.
```

Files to build

File	Purpose
`app/api/admin/refunds/paymongo/process/route.ts`	POST endpoint. Reads a pending refund row, calls PayMongo `POST /v1/refunds`, stores the returned `refund_id`, moves our row status to `processing`. Guards for age (180 day PayMongo limit), amount (can't exceed original charge), duplicate calls.
`app/api/paymongo/webhook/route.ts`	New handlers for `refund.succeeded` and `refund.failed` events. Look up our row by stored `paymongo_refund_id`. On success, mark `completed` and fire downstream side-effects. On failure, mark `failed` with the reason.
`app/admin/refunds/page.tsx`	PROCESS VIA PAYMONGO button on each pending row. Hard confirm dialog citing the exact amount. Show processing state while PayMongo works. Auto-hides on ineligible rows (older than 180 days).

Behavior

- Original charge older than 180 days: button disabled, tooltip explains PayMongo's window.
- Amount validation: reject if refund amount exceeds original charge total.
- Partial refunds supported — already how downgrades work.
- PayMongo failure: row moves to `failed` with the reason from PayMongo's response. Admin gets an in-app notification to follow up.
- Every action audit-logged with actor, target, amount, timestamp.

Step 2 — Ticket cancel flow

Why: answers the "pag ni-cancel yung ticket" workflow. Currently cancelling a ticket is a manual DB flip with no follow-through — no refund, no waitlist promotion, no email to the member.

Current flow (5 manual actions)

```
1. Admin edits event_tickets row in the DB → status=cancelled
2. Admin opens PayMongo dashboard, finds the payment, clicks refund
3. Admin returns to /admin/refunds, adds a refund row, marks completed
4. Admin manually emails the member
5. Waitlist entries stay waiting even though a seat opened
```

After step 2 (1 click)

```
Admin clicks CANCEL on a ticket row
  → confirm dialog
  → orchestrator fires in sequence:
      - event_tickets.status = 'cancelled'
      - PayMongo refund initiated (via step 1's endpoint)
      - promoteNextWaitlist(eventId, 1) — next in line notified
      - branded cancellation email sent to the member
      - capacity re-opened (if event was full)
  → refund webhook eventually confirms → refund row completed
```

Files to build

File	Purpose
`app/api/admin/events/tickets/cancel/route.ts`	Orchestrator. Loads the ticket + original payment, marks it cancelled, calls step 1's refund endpoint, calls `promoteNextWaitlist`, queues the cancellation email, audit-logs.
`lib/emails/ticket-cancelled.ts`	Branded shell (same layout as event-ticket, welcome, session-alert). Subject: "Your ticket for [event] has been cancelled." Body cites the refund amount and expected timing.
`app/admin/events/[id]/tickets/page.tsx`	CANCEL button per row. Confirm dialog. Optimistic UI update.

Step 3 — Member notifications on refund lifecycle

Why: today a member submits a downgrade request and hears nothing until they check back on the ticket page.

What to wire: reuse existing `notifications` table + `lib/emails/*` shells + step 1's refund webhook.

Event	In-app notif	Email
Refund created (queued)	"Your refund request is being processed."	Optional — usually silent
Refund succeeded	"Your ₱X refund has been sent."	"Refund confirmed — expect it in your original payment method within 3–5 business days."
Refund failed	"We couldn't process your refund automatically."	"Our team will follow up within 24h to resolve this."
Tier upgrade paid	"Your ticket has been upgraded to [Tier]. New QR is ready."	Reuse existing event ticket confirmation with a "tier upgraded" line

All four hook off the same PayMongo webhook + refund status transitions from step 1. No new plumbing.

Step 4 — Refund performance panel

Why: a small quality-of-life polish once volume warrants. Not urgent.

On `/admin/refunds`:

- Header tiles: total refunded this month, pending count, failed-needs-followup count.
- Stale flag on any row that's been `pending` for more than 7 days.
- Reason breakdown: tier downgrade / cancellation / order / donation.
- CSV export for accounting reconciliation.

Parked (out of this track)

- **Vouchers / store credit** — alternative to real refunds for downgrades. Fan-friendly, larger scope. Revisit if PayMongo refunds prove unreliable.
- **Programmable no-refund windows** — e.g. no refunds < 48h before an event. Simple flag on events, but not a real problem until we see abuse.
- **Refund fee reconciliation** — PayMongo takes a cut on the original charge; who eats it on refund? Answer once we have volume.
- **Deeper finance analytics** — cohort views, per-event refund rates. Step 4's tiles are the MVP; deeper reporting waits.

Sequence of work

1. Email PayMongo support to confirm Refunds API is enabled. Apply the pending refunds migration on the remote database.
2. Build step 1. Ship and test on a single low-value refund end-to-end.
3. Build step 2 using step 1's endpoint. Ship and test on one recent cancellation.
4. Wire step 3's notifications alongside 1 and 2 — no separate deploy needed.
5. Step 4 when refund volume warrants a dashboard.